

CPP - Module 01

Memoria, punteros y referencias — de cero a entenderlo todo de verdad

INDICE

PARTE 1 — La memoria: stack y heap

1. El mapa de la memoria RAM de tu programa
2. El stack — la pila de bandejas
3. El heap — el almacen con taquillas
4. Por que existen los dos — cuando usar cada uno

PARTE 2 — new y delete 5. El problema de crear objetos fuera de su funcion 6. new — reservar en el heap y llamar al constructor 7. delete — destruir y liberar 8. Los tres errores mortales de la memoria dinamica

PARTE 3 — new[] y delete[] 9. El problema de crear N objetos de golpe 10. new[] — un array entero en el heap 11. delete[] — por que los corchetes son obligatorios 12. El constructor por defecto y new[]

PARTE 4 — Punteros y referencias 13. Repaso de punteros — el cajon de la direccion 14. El operador & en sus dos significados — la confusion mas comun 15. El operador * en sus dos significados 16. Las referencias — el segundo nombre 17. Las tres reglas inamovibles de las referencias 18. Puntero vs referencia — la tabla definitiva 19. Cuando usar puntero y cuando referencia

PARTE 5 — Referencias y punteros en clases 20. El problema — HumanA siempre tiene arma, HumanB no 21. HumanA — referencia porque nunca esta desarmado 22. HumanB — puntero porque puede estarlo 23. Por que las referencias de clase van en la lista de inicializacion 24. La prueba: el arma cambia y attack() lo ve

PARTE 6 — Streams de ficheros 25. El problema — leer y escribir archivos 26. ifstream — abrir un fichero para leer 27. ofstream — crear un fichero para escribir 28. El bucle de lectura linea a linea 29. El algoritmo de reemplazar sin usar replace

PARTE 7 — Punteros a funciones miembro 30. El problema — el bosque de if/else 31. Punteros a funciones normales — el repaso base 32. Punteros a funciones miembro — la diferencia clave 33. La sintaxis desglosada simbolo por simbolo 34. Arrays de punteros — el patron del ex05

PARTE 8 — El switch con fall-through 35. El switch — que es y como funciona 36. El break — por que es obligatorio casi siempre 37. El fall-through intencional — la clave del ex06

PARTE 9 — Los ejercicios explicados uno a uno 38. Ex00: BrailiiiiinnzzzZ 39. Ex01: Moar brainz! 40. Ex02: HI THIS IS BRAIN 41. Ex03: Unnecessary violence 42. Ex04: Sed is for losers 43. Ex05: Harl 2.0 44. Ex06: Harl filter

Glosario completo

PARTE 1 — LA MEMORIA: STACK Y HEAP

1. El mapa de la memoria RAM de tu programa

Cuando ejecutas un programa, el sistema operativo le asigna un bloque de memoria RAM. Ese bloque no es una masa homogenea — esta organizado en zonas con reglas distintas.

```
MEMORIA RAM de tu programa
+-----+ <- direccion alta
|      STACK      | crece hacia abajo cuando entras en funciones
|      v          |
|                 |
| (espacio libre) |
|                 |
|      ^          |
|      HEAP       | crece hacia arriba cuando usas new
+-----+
| variables globales | viven toda la ejecucion
+-----+
| codigo (.text)     | las funciones compiladas — solo lectura
+-----+ <- direccion baja
```

Las **funciones** viven en .text desde que el programa arranca hasta que termina. No se crean ni destruyen durante la ejecucion — estan escritas en piedra.

Los **datos** (variables, objetos) viven en el stack o en el heap, y eso cambia todo.

2. El stack — la pila de bandejas

Imagina la cafetería de 42. Las bandejas están apiladas. Cuando entras a comer (entras en una función), coges una bandeja nueva y la pones encima. En esa bandeja metes tu comida (tus variables locales). Cuando terminas (la función termina), devuelves la bandeja — y todo lo que había en ella desaparece automáticamente.

```
main() llama a randomChump("Bob"):
```

```
+-----+
| Zombie z("Bob") | <- bandeja de randomChump()
+-----+
+-----+
| int x = 5       | <- bandeja de main()
+-----+
```

```
randomChump() termina — su bandeja se retira:
```

```
+-----+
| int x = 5       | <- bandeja de main() sigue
+-----+
Zombie z ya no existe
```

Las reglas del stack:

- **Automatico** — el programa gestiona la memoria solo. Tú no haces nada.
- **Rapido** — añadir o quitar una bandeja es una sola instrucción del procesador.
- **Limitado** — el stack tiene un tamaño máximo fijo (1-8 MB según el sistema). Si te pasas, el programa peta. Eso se llama **stack overflow** — sí, igual que la web.
- **LIFO** — Last In, First Out. La última bandeja que entro es la primera que sale.
- **El objeto muere al salir del scope { }** — sin excepciones.

3. El heap — el almacén con taquillas

Imagina un almacén enorme con miles de taquillas numeradas. Cuando usas `new`, el sistema te busca una taquilla libre, te la asigna, y te da la llave — el puntero. Lo que metes en esa taquilla **vive indefinidamente** — no le importa si las funciones entran y salen. Solo desaparece cuando tú mismo la abres y la vacías con `delete`.

HEAP (el almacen):

```
Taquilla 0x7f3a: [Zombie "Alice"]  <- newZombie creo esto
Taquilla 0x7f4b: [ vacia ]
Taquilla 0x7f5c: [ vacia ]
```

STACK (las llaves):

```
main():
  Zombie *z = 0x7f3a  <- la llave de la taquilla de Alice
```

La variable `z` vive en el stack — es solo un numero, una direccion. El **objeto real** vive en el heap. Cuando `z` desaparece del stack (porque la funcion termina), la taquilla del heap **sigue existiendo**. Nadie la vacio. Eso es un **memory leak**.

Las reglas del heap:

- **Manual** — tu decides cuando nace (`new`) y cuando muere (`delete`).
- **Mas lento** — el sistema tiene que buscar un hueco libre, marcarlo, actualizar su mapa.
- **Grande** — puede ser de gigabytes.
- **Sin scope** — el objeto vive hasta que haces `delete` , independientemente de las funciones.

4. Por que existen los dos — cuando usar cada uno

Antes de escribir una sola linea, hazte esta pregunta:

Necesito que este objeto viva fuera de la funcion donde lo creo?

NO	SI
STACK	HEAP
Zombie z(name)	Zombie *z = new Zombie(name)
z.announce()	z->announce()
} <- muere solo	delete z <- tu decides cuando

Usa el stack cuando el objeto solo lo necesitas dentro de esa funcion. **Usa el heap cuando** el objeto necesita sobrevivir a la funcion que lo creo.

Grabate esto: **usa siempre el stack por defecto. Solo saltas al heap cuando el stack no**

te sirve.

PARTE 2 — NEW Y DELETE

5. El problema de crear objetos fuera de su funcion

Imagina que tienes esta funcion:

```
Zombie *crearZombie(std::string nombre) {  
    Zombie z(nombre);    // z vive en el stack de esta funcion  
    return &z;           // devuelves la direccion de z  
}                        // z MUERE aqui — su bandeja se retira
```

Que pasa cuando alguien usa el puntero que devolviste?

```
Zombie *ptr = crearZombie("Alice");  
ptr->announce();    // PELIGRO — ptr apunta a un cajon que ya fue devuelto
```

Trazando en memoria:

```
Durante crearZombie():  
    bandeja de crearZombie: [Zombie "Alice" en direccion 0x7fff5a]  
    return &z -> devuelve 0x7fff5a
```

```
Despues de crearZombie():  
    bandeja de crearZombie fue retirada  
    la direccion 0x7fff5a ya no pertenece a nadie  
    ptr = 0x7fff5a -> DANGLING POINTER
```

Eso se llama **dangling pointer**. El compilador no avisa. Puede crashear, dar resultados incorrectos, o aparentemente funcionar. La solucion: crear el objeto en el **heap**.

6. new — reservar en el heap y llamar al constructor

`new` hace tres cosas a la vez:

1. Busca una taquilla libre en el heap del tamaño necesario

2. Llama al **constructor** del objeto con los parametros que le das

3. Devuelve un **puntero** al objeto ya inicializado

```
Zombie *z = new Zombie("Alice");
```

Trazando:

```
new Zombie("Alice"):  
  1. Busca taquilla libre en el heap -> 0x7f3a  
  2. Llama a Zombie::Zombie("Alice")  
    -> _name = "Alice"  
    -> imprime "zombie Alice created"  
  3. Devuelve 0x7f3a  
  
z = 0x7f3a    <- z guarda la llave de la taquilla
```

Para acceder a los miembros de un objeto creado con `new`, usas `->`:

```
z->announce();    // -> porque z es un puntero  
(*z).announce(); // equivalente - desreferencias primero, luego accedes
```

7. delete — destruir y liberar

`delete` hace dos cosas a la vez:

1. Llama al **destructor** del objeto

2. Libera la taquilla en el heap

```
delete z;
```

Trazando:

```
delete z (donde z = 0x7f3a):  
  1. Llama a Zombie::~~Zombie()  
    -> imprime "Zombie Alice destroyed"  
  2. Marca la taquilla 0x7f3a como libre  
    z sigue siendo 0x7f3a (la direccion no desaparece del stack)  
    pero la taquilla ya fue vaciada -> z es ahora un DANGLING POINTER
```

La buena practica: poner el puntero a `nullptr` despues del `delete`:

```
delete z;  
z = nullptr;
```

Las parejas obligatorias — nunca las mezcles:

```
new      -> delete  
new[]    -> delete[]  
malloc   -> free
```

8. Los tres errores mortales de la memoria dinamica

Memory leak — la taquilla olvidada:

```
void funcion() {  
    Zombie *z = new Zombie("Alice");  
    z->announce();  
    // olvidamos el delete  
} // z desaparece del stack pero Alice sigue en el heap sin llave  
// MEMORY LEAK — nadie puede liberar esa memoria nunca
```

Dangling pointer — la llave de una taquilla vacia:

```
Zombie *z = new Zombie("Alice");  
delete z;  
z->announce(); // PELIGRO — la taquilla fue vaciada
```

Double free — abrir una taquilla ya abierta:

```
Zombie *z = new Zombie("Alice");  
delete z;  
delete z; // PELIGRO — el heap manager se corrompe
```

PARTE 3 — NEW[] Y DELETE[]

9. El problema de crear N objetos de golpe

El ex01 pide “a single allocation” — una sola reserva de memoria para N zombies. Para eso existe `new[]` .

10. `new[]` — un array entero en el heap

```
Zombie *horde = new Zombie[N];
```

Esto hace tres cosas:

1. Reserva espacio en el heap para N objetos Zombie **consecutivos** — una sola taquilla grande
2. Llama al **constructor por defecto** de cada uno de los N zombies
3. Devuelve un puntero al **primero**

Visualizando en el heap:

```
HEAP:
+-----+-----+-----+-----+-----+
| Zombie 0 | Zombie 1 | Zombie 2 | Zombie 3 | Zombie 4 |
+-----+-----+-----+-----+-----+
      ^
horde apunta aqui (al primero)
```

Acceder a los elementos — igual que un array normal de C:

```
horde[0].announce(); // punto — horde[0] es un objeto, no un puntero
horde[2].announce();
for (int i = 0; i < N; i++)
    horde[i].announce();
```

11. `delete[]` — por que los corchetes son obligatorios

```
delete[] horde; // los [] son OBLIGATORIOS
```

El `[]` le dice al compilador: “esto es un array entero. Destruye **todos** los elementos”.

Sin los corchetes:


```
delete horde;    // MAL – solo destruye el primer zombie
```

Trazando que pasa con y sin corchetes (N=5):

```
delete[] horde:    destructor de Zombie 4, 3, 2, 1, 0 (inverso)
                   libera toda la taquilla de golpe OK
```

```
delete horde:      destructor de Zombie 0 solamente
                   los zombies 1-4 nunca se destruyen MAL
                   MEMORY LEAK garantizado
```

12. El constructor por defecto y new[]

Cuando haces `new Zombie[N]`, el compilador llama al constructor **sin parametros** de cada elemento. Si solo tienes:

```
Zombie(std::string name);    // exige un nombre
```

El compilador al intentar `new Zombie[N]` te grita:

```
error: no matching constructor for initialization of 'Zombie'
```

La solucion: anadir un constructor por defecto:

```
Zombie::Zombie() {
    // _name queda como "" (string vacio por defecto)
}
```

Despues del `new Zombie[N]`, recorres el array y asignas el nombre con `setName`:

```
Zombie *zombieHorde(int N, std::string name) {
    Zombie *horde = new Zombie[N];           // N constructores por defecto
    for (int i = 0; i < N; i++)
        horde[i].setName(name);              // les pones el nombre
    return horde;
}
```

PARTE 4 — PUNTEROS Y REFERENCIAS

13. Repaso de punteros — el cajon de la direccion

Un puntero es una variable que en lugar de guardar un valor normal, guarda la **direccion** de otro cajon:

```
std::string brain = "HI THIS IS BRAIN";
std::string *ptr  = &brain;
```

En memoria:

```
+-----+-----+
| 1040 | "HI THIS IS BRAIN" | <- cajon de brain
+-----+-----+
+-----+-----+
| 2080 | 1040      | <- cajon de ptr (guarda la direccion 1040)
+-----+-----+
```

14. El operador & en sus dos significados — la confusion mas comun

El simbolo & hace cosas completamente distintas segun donde aparece.

Significado 1 — en una expresion: "dame la direccion de este cajon"

```
std::string brain = "HI THIS IS BRAIN";
std::string *ptr = &brain;
//                ^
//                operador de direccion - "dame la direccion del cajon de brain"
```

Significado 2 — en una declaracion: "esto es una referencia"

```
std::string &ref = brain;
//          ^
//  parte del tipo - "ref es una referencia a string"
```

La clave para distinguirlos:

El & pertenece a la variable (declaracion)? -> tipo referencia

El & pertenece a otra variable (expresion)? -> operador de direccion

```
std::string &ref = brain;    -> el & pertenece a ref    -> tipo referencia
```

```
std::string *ptr = &brain;   -> el & pertenece a brain -> operador de direccion
```

15. El operador * en sus dos significados

Significado 1 — en una declaracion: “esto es un puntero”

```
std::string *ptr = &brain;
//           ^
//  parte del tipo — "ptr es un puntero a string"
```

Significado 2 — en una expresion: “ve a esa direccion y dame lo que hay”

```
*ptr    // va a la direccion que contiene ptr (1040) y devuelve "HI THIS IS BRAIN"

std::cout << *ptr << std::endl;    // imprime "HI THIS IS BRAIN"
*ptr = "otro texto";               // modifica el cajon 1040
std::cout << brain << std::endl;    // "otro texto" — brain cambio
```

16. Las referencias — el segundo nombre

Una referencia es un **segundo nombre** para exactamente el mismo cajon. No hay cajon nuevo — es el mismo cajon con dos etiquetas pegadas encima.

```
std::string brain = "HI THIS IS BRAIN";
std::string &ref  = brain;
```

En memoria:

```
+-----+-----+
| 1040 | "HI THIS IS BRAIN" |
+-----+-----+
      ^           ^
    brain       ref
(misma direccion, dos nombres)
```

brain y ref son el mismo cajon. Si cambias ref, cambia brain:

```
ref = "otro texto";  
std::cout << brain << std::endl;    // "otro texto"
```

El ex02 (HI THIS IS BRAIN) demuestra esto — imprime las tres direcciones y son **todas iguales**:

```
Direccion de brain:    0x7fff5a00  
Direccion de ptr:      0x7fff5a00    <- misma  
Direccion de ref:      0x7fff5a00    <- misma
```

Hay un solo cajon en memoria. Tres formas de acceder a el.

17. Las tres reglas inamovibles de las referencias

Regla 1 — Debes inicializarla al declararla:

```
std::string &ref;           // ERROR — segunda etiqueta de que cajon?  
std::string &ref = brain;   // correcto
```

Regla 2 — Una vez asignada, no puede apuntar a otro cajon:

```
std::string a = "Alice";  
std::string b = "Bob";  
std::string &ref = a;    // ref es segunda etiqueta del cajon de a  
  
ref = b;    // NO hace que ref apunte al cajon de b  
            // COPIA el contenido de b en el cajon de a  
            // a ahora vale "Bob"  
            // ref sigue siendo etiqueta del cajon de a
```

Trazando:

```
Antes:  
    cajon 1040: "Alice"    <- a, ref  
    cajon 2080: "Bob"     <- b  
  
ref = b -> copia el contenido de 2080 en 1040:
```

Despues:

```
cajon 1040: "Bob"      <- a, ref (mismo cajon, nuevo contenido)
cajon 2080: "Bob"      <- b (sin cambios)
```

Regla 3 — No puede ser nula:

```
std::string &ref = nullptr;    // ERROR — siempre tiene que apuntar a un cajon rea
```

18. Puntero vs referencia — la tabla definitiva

	Puntero	Referencia
Declaracion	<code>std::string *p = &brain</code>	<code>std::string &r = brain</code>
Tiene cajon propio	Si — guarda la direccion	No — es el mismo cajon
Puede ser nulo	Si — <code>nullptr</code>	No — siempre apunta a algo
Puede reasignarse	Si — puede apuntar a otro cajon	No — siempre el mismo
Necesita inicializacion	No (aunque deberias)	Si — obligatorio
Acceder al valor	<code>*p</code> o <code>p->miembro</code>	<code>r</code> o <code>r.miembro</code>
Obtener direccion	<code>p</code> (ya es la direccion)	<code>&r</code> (igual que <code>&brain</code>)

19. Cuando usar puntero y cuando referencia

La pregunta es simple:

El cajon al que apuntas puede no existir (ser null)?

NO	SI
REFERENCIA	PUNTERO
(siempre apunta a algo)	(puede ser <code>nullptr</code>)

Usa referencia cuando:

- El objeto siempre va a existir — nunca sera nulo
- No necesitas reasignar a que cajon apunta

- Quieres código más limpio (sin * ni ->)

Usa puntero cuando:

- El objeto puede no existir — necesitas nullptr como valor válido
 - Necesitas poder cambiar a qué cajón apunta
 - Necesitas arrays dinámicos
-

PARTE 5 — REFERENCIAS Y PUNTEROS EN CLASES

20. El problema — HumanA siempre tiene arma, HumanB no

El ex03 te plantea una pregunta:

"In which case do you think it would be best to use a pointer to Weapon? And a reference? Why?"

- **HumanA** recibe el arma en el constructor — **siempre tiene arma**.
- **HumanB** no recibe arma en el constructor — **puede no tener arma**.

Esta diferencia dicta la elección:

HumanA siempre tiene arma -> el arma nunca será null -> REFERENCIA
HumanB puede no tener arma -> el arma puede ser null -> PUNTERO

21. HumanA — referencia porque nunca está desarmado

```
class HumanA {
    private:
        std::string _name;
        Weapon      &_weapon;    // REFERENCIA — nunca es nula
    public:
        HumanA(std::string name, Weapon &weapon);
        void attack(void);
};
```

```
HumanA::HumanA(std::string name, Weapon &weapon) : _name(name), _weapon(weapon) {
```

```

        // _weapon DEBE inicializarse en la lista
    }

void HumanA::attack(void) {
    std::cout << _name << " attacks with their " << _weapon.getType() << std::endl
    //
    //
    // punto - _weapon es una referencia (objeto dir
}

```

22. HumanB — puntero porque puede estar desarmado

```

class HumanB {
private:
    std::string _name;
    Weapon      *_weapon;    // PUNTERO - puede ser nullptr
public:
    HumanB(std::string name);
    void setWeapon(Weapon &weapon);
    void attack(void);
};

HumanB::HumanB(std::string name) : _name(name), _weapon(NULL) {
    // _weapon empieza como nullptr
}

void HumanB::setWeapon(Weapon &weapon) {
    _weapon = &weapon;    // guarda la direccion del arma
}

void HumanB::attack(void) {
    if (!_weapon) {
        std::cout << _name << " has no weapon" << std::endl;
        return ;
    }
    std::cout << _name << " attacks with their " << _weapon->getType() << std::en
    //
    //
    // flecha - _weapon es un puntero
}

```

23. Por que las referencias de clase van en la lista de

inicializacion

Regla sin excepciones en C++:

Los atributos que son referencias DEBEN inicializarse en la lista de inicializacion.

```
// MAL - no compila
HumanA::HumanA(std::string name, Weapon &weapon) {
    _name    = name;
    _weapon = weapon;    // ERROR - no puedes asignar una referencia en el cuerpo
}

// BIEN
HumanA::HumanA(std::string name, Weapon &weapon) : _name(name), _weapon(weapon) {
```

Una referencia es una segunda etiqueta — debe existir desde el momento en que el cajon existe. No puedes construir el cajon vacio y pegar la etiqueta despues.

24. La prueba: el arma cambia y attack() lo ve

```
Weapon club = Weapon("crude spiked club");
HumanA bob("Bob", club);
bob.attack();                                // "Bob attacks with their crude spike
club.setType("some other type of club");
bob.attack();                                // "Bob attacks with their some other
```

Por que el segundo attack() ve el cambio? Porque `_weapon` es una **referencia** al objeto `club` original.

Trazando:

```
club._type -> cajon 1040: "crude spiked club"
               ^
bob._weapon ----- | (referencia al cajon de club)

club.setType("some other type of club"):
    cajon 1040: "some other type of club"

bob.attack() -> _weapon.getType() -> lee cajon 1040 -> "some other type of club"
```

Si `_weapon` fuera una copia del arma, bob tendria su propia copia de "crude spiked club" que nunca cambiaria.

PARTE 6 — STREAMS DE FICHEROS

25. El problema — leer y escribir archivos

El ex04 pide abrir un fichero, reemplazar todas las ocurrencias de una cadena por otra, y guardar el resultado en un fichero nuevo. En C++ usas **streams de fichero**:

```
#include <fstream>
```

Los tres tipos:

Tipo	Para que
std::ifstream	Leer desde fichero
std::ofstream	Escribir en fichero
std::fstream	Leer y escribir

26. ifstream — abrir un fichero para leer

```
std::ifstream entrada("datos.txt");

if (!entrada.is_open()) {
    std::cerr << "Error: no se puede abrir el fichero" << std::endl;
    return 1;
}
```

`is_open()` devuelve true si el fichero se abrió correctamente. Siempre compruébalo.

En C++98, los constructores de `ifstream` aceptan `const char *`, no `std::string`. Necesitas convertir:

```
std::string nombre = "datos.txt";
std::ifstream entrada(nombre.c_str()); // .c_str() convierte a const char *
```

27. ofstream — crear un fichero para escribir

```
std::string nombreSalida = std::string(argv[1]) + ".replace";
std::ofstream salida(nombreSalida.c_str());

if (!salida.is_open()) {
    std::cerr << "Error: no se puede crear el fichero" << std::endl;
    entrada.close();
    return 1;
}
```

Escribir en el es identico a cout:

```
salida << linea_modificada << "\n";
```

28. El bucle de lectura linea a linea

```
std::string linea;
while (std::getline(entrada, linea)) {
    // procesa la linea
    salida << linea << "\n";
}
entrada.close();
salida.close();
```

`std::getline` lee una línea hasta el `'\n'` (sin incluirlo). Devuelve el stream — que se convierte a false cuando llega al final del fichero. Asi el while termina automaticamente.

29. El algoritmo de reemplazar sin usar replace

El subject prohíbe `std::string::replace`. Usas `find`, `erase` e `insert`.

El patron para TODAS las ocurrencias:

```
size_t pos = 0;
while ((pos = linea.find(s1, pos)) != std::string::npos) {
    linea.erase(pos, s1.length());
    linea.insert(pos, s2);
    pos += s2.length();    // esta linea es critica
}
```

```
}
```

Por que `pos += s2.length()` ? Sin avanzar el cursor, puedes entrar en bucle infinito.
Imagina `s1="aa"` y `s2="aaa"`:

```
Antes: "aaa mundo"
find("aa", 0) -> pos = 0
erase(0, 2) -> "a mundo"
insert(0, "aaa") -> "aaaa mundo"
find("aa", 0) -> pos = 0  <- encuentra de nuevo en el texto recién insertado
-> bucle infinito!
```

Con `pos += s2.length()` :

```
Despues del insert: "aaaa mundo"
pos += 3 -> pos = 3
find("aa", 3) -> busca desde posicion 3
-> no encuentra mas "aa" -> termina OK
```

PARTE 7 — PUNTEROS A FUNCIONES MIEMBRO

30. El problema — el bosque de if/else

El ex05 tiene cuatro mensajes: DEBUG, INFO, WARNING, ERROR. La solución obvia:

```
void Harl::complain(std::string level) {
    if (level == "DEBUG")
        debug();
    else if (level == "INFO")
        info();
    else if (level == "WARNING")
        warning();
    else if (level == "ERROR")
        error();
}
```

El subject lo prohíbe: “without using a forest of if/else if”. La solución usa **punteros a funciones miembro**.

31. Punteros a funciones normales — el repaso base

Las funciones tienen una direccion en memoria. Puedes guardarla en un puntero y llamar a la funcion a traves de el:

```
void saludar() { std::cout << "Hola" << std::endl; }
void despedir() { std::cout << "Adios" << std::endl; }

void (*ptr)() = saludar;    // ptr guarda la direccion de saludar
ptr();                      // llama a saludar()
ptr = despedir;
ptr();                      // llama a despedir()
```

La sintaxis `void (*ptr)()` se lee:

- `void` — la funcion devuelve void
 - `(*ptr)` — ptr es un puntero (parentesis obligatorios)
 - `()` — la funcion no recibe parametros
-

32. Punteros a funciones miembro — la diferencia clave

Los metodos de clase tienen un parametro oculto: `this`. Un puntero a funcion normal no sabe nada de clases. Para apuntar a un metodo necesitas un **puntero a funcion miembro**:

```
Puntero a funcion normal:
direccion -> codigo de la funcion
```

```
Puntero a funcion miembro:
direccion -> codigo del metodo
+ informacion de que pertenece a Harl (para gestionar this)
```

33. La sintaxis desglosada simbolo por simbolo

```
void (Harl::*ptr)(void);
```

Desglosado:

- `void` — el metodo devuelve void

- (Harl::*ptr) — ptr es un puntero a algo de la clase Harl
- (void) — el metodo no recibe parametros

Comparacion con puntero a funcion normal:

```
void (*ptr)(void);           // puntero a funcion normal
void (Harl::*ptr)(void);     // puntero a funcion miembro de Harl
//          ^^^^^
//   la unica diferencia — Harl::
```

Asignar un metodo:

```
ptr = &Harl::debug; // el & es obligatorio en el estandar
```

Llamar al metodo — necesitas un objeto:

```
Harl harl;
(harl.*ptr)(); // con objeto directo — parentesis OBLIGATORIOS
```

Desde dentro de la propia clase:

```
(this->*ptr)();
```

34. Arrays de punteros — el patron del ex05

```
void Harl::complain(std::string level) {
    std::string niveles[4] = { "DEBUG", "INFO", "WARNING", "ERROR" };

    void (Harl::*metodos[4])(void) = {
        &Harl::debug,
        &Harl::info,
        &Harl::warning,
        &Harl::error
    };

    for (int i = 0; i < 4; i++) {
        if (niveles[i] == level) {
            (this->*metodos[i])();
            return;
        }
    }
}
```

```
}
```

Trazando con level = "WARNING":

```
i=0: "DEBUG" == "WARNING"? NO
i=1: "INFO" == "WARNING"? NO
i=2: "WARNING" == "WARNING"? SI
    -> (this->*metodos[2])() -> llama a this->warning()
    -> return
```

Sin un solo if/else para elegir el metodo.

PARTE 8 — EL SWITCH CON FALL-THROUGH

35. El switch — que es y como funciona

El switch selecciona entre multiples caminos segun el valor de una expresion entera:

```
int nivel = 2;

switch (nivel) {
    case 0:
        std::cout << "DEBUG" << std::endl;
        break;
    case 1:
        std::cout << "INFO" << std::endl;
        break;
    case 2:
        std::cout << "WARNING" << std::endl;
        break;
    case 3:
        std::cout << "ERROR" << std::endl;
        break;
    default:
        std::cout << "Desconocido" << std::endl;
        break;
}

// imprime: WARNING
```

Limitacion importante: el switch solo acepta tipos enteros (int, char, enum). Los strings

NO funcionan con switch. Por eso en el ex06 tienes que convertir el nivel (string) a un numero primero.

36. El break — por que es obligatorio casi siempre

Sin break, la ejecucion **cae** al siguiente case:

```
int nivel = 1;
switch (nivel) {
    case 0:
        std::cout << "DEBUG" << std::endl;
        break;
    case 1:
        std::cout << "INFO" << std::endl;
        // sin break — cae al siguiente
    case 2:
        std::cout << "WARNING" << std::endl;
        break;
}
// imprime: INFO y luego WARNING
```

Trazando:

```
nivel = 1 -> entra en case 1
    imprime "INFO"
    no hay break -> no sale del switch
    cae a case 2 (ignora la etiqueta case 2: — es solo una etiqueta)
    imprime "WARNING"
    break -> sale del switch
```

Eso se llama **fall-through**. Normalmente es un error. En el ex06 es intencional.

37. El fall-through intencional — la clave del ex06

El ex06 pide imprimir **desde el nivel seleccionado en adelante**. El fall-through sin break lo hace de forma natural:

```
void Harl::filterComplain(std::string level) {
    int index = getLevelIndex(level);

    switch (index) {
```

```

    case 0:
        debug();
        // sin break -> cae a INFO
    case 1:
        info();
        // sin break -> cae a WARNING
    case 2:
        warning();
        // sin break -> cae a ERROR
    case 3:
        error();
        break;
    default:
        std::cout << "[ Probably complaining about insignificant problems ]"
                  << std::endl;
        break;
}
}

```

Trazando con level = "WARNING" (index = 2):

```

switch(2):
    case 0: NO — salta
    case 1: NO — salta
    case 2: SI — entra
        warning() -> imprime [ WARNING ] y su mensaje
        sin break -> cae
    case 3:
        error() -> imprime [ ERROR ] y su mensaje
        break -> sale

```

El punto de entrada es el nivel seleccionado y desde ahí todo cae hacia abajo automáticamente.

PARTE 9 — LOS EJERCICIOS EXPLICADOS UNO A UNO

38. Ex00: BraiiiiiiinnnzzzzZ

Concepto central: stack vs heap — cuando muere cada zombie.

La demostracion clave esta en el orden de los destructores:

```
main():
    heapZombie = newZombie("Alice")    -> constructor
    heapZombie->announce()
    randomChump("Bob")                  -> constructor + announce + DESTRUCTOR (muere
    delete heapZombie                    -> DESTRUCTOR (tu decides cuando)
```

Bob muere antes de que el main decida borrar a Alice. Eso es la diferencia stack/heap.

39. Ex01: Moar brainz!

Concepto central: new[] y delete[] — un array de objetos en el heap.

```
Zombie *zombieHorde(int N, std::string name) {
    Zombie *horde = new Zombie[N];    // N constructores por defecto
    for (int i = 0; i < N; i++)
        horde[i].setName(name);
    return horde;
}

// en main:
Zombie *horde = zombieHorde(5, "Legion");
for (int i = 0; i < 5; i++)
    horde[i].announce();
delete[] horde;    // [] OBLIGATORIOS
```

40. Ex02: HI THIS IS BRAIN

Concepto central: las tres formas de acceder al mismo cajon.

```
std::string brain    = "HI THIS IS BRAIN";
std::string *stringPTR = &brain;
std::string &stringREF = brain;

// Las tres direcciones son IGUALES
std::cout << &brain    << std::endl;
std::cout << stringPTR  << std::endl;
std::cout << &stringREF << std::endl;

// Los tres valores son IGUALES
```

```
std::cout << brain      << std::endl;
std::cout << *stringPTR << std::endl;
std::cout << stringREF  << std::endl;
```

41. Ex03: Unnecessary violence

Concepto central: referencia vs puntero como atributo de clase.

- `Weapon` — tiene `_type`, `getType()` con `const std::string &`, `setType()`
 - `HumanA` — `Weapon &_weapon` — siempre armado — usa punto para acceder
 - `HumanB` — `Weapon *_weapon` — puede estar desarmado — usa flecha, comprueba null
-

42. Ex04: Sed is for losers

Concepto central: streams de ficheros + reemplazo sin replace.

Flujo: comprobar argumentos -> abrir fichero de entrada -> crear fichero.replace -> leer linea a linea -> reemplazar s1 por s2 -> escribir -> cerrar.

43. Ex05: Harl 2.0

Concepto central: punteros a funciones miembro — sin if/else.

Array de strings paralelo al array de punteros. El indice i funciona como llave para ambos arrays.

44. Ex06: Harl filter

Concepto central: switch con fall-through intencional.

El ejecutable se llama `harlFilter` (nombre específico). Convierte el string del argv a indice, entra en el switch por ese indice, y la cascada sin break imprime desde ese nivel hasta ERROR.

GLOSARIO COMPLETO

Termino	Definicion
Stack	Zona de memoria automatica — como bandejas apiladas, se retiran solas al salir de la funcion
Heap	Zona de memoria dinamica — como taquillas, vive hasta que tu la vacias
Stack frame	El espacio que ocupa una funcion en el stack — sus variables locales
new	Reserva taquilla en el heap + llama al constructor
delete	Llama al destructor + libera la taquilla
new[]	Reserva un array en el heap + llama al constructor por defecto de cada elemento
delete[]	Llama al destructor de cada elemento + libera todo el array
Memory leak	Taquilla reservada en el heap que nunca se libera — la llave se perdio
Dangling pointer	Puntero que apunta a una taquilla ya vaciada — usarlo es comportamiento indefinido
Double free	Llamar a delete dos veces sobre el mismo puntero — corrompe el heap
. (punto)	Accede a un miembro de un objeto directo (no puntero)
-> (flecha)	Accede a un miembro a traves de un puntero
nullptr	Puntero nulo — no apunta a ninguna taquilla valida
Referencia	Segunda etiqueta del mismo cajon — no tiene cajon propio
& en expresion	Operador de direccion — “dame el numero del cajon de esta variable”
& en declaracion	Tipo referencia — “esta variable es una segunda etiqueta de otro cajon”
* en expresion	Desreferenciar — “ve a esa direccion y dame lo que hay”
* en declaracion	Tipo puntero — “esta variable guarda la direccion de otro cajon”
Fall-through	En un switch sin break, la ejecucion cae al siguiente case

Fall-through intencional	Usar el fall-through a proposito para ejecutar multiples cases en cascada
Puntero a funcion	Variable que guarda la direccion de una funcion libre
Puntero a funcion miembro	Variable que guarda la direccion de un metodo de clase
void (Clase::*ptr)()	Declaracion de puntero a funcion miembro de Clase
(this->*ptr)()	Llamar al metodo apuntado por ptr desde dentro de la clase
std::ifstream	Stream de lectura de fichero
std::ofstream	Stream de escritura de fichero
is_open()	Comprueba si el fichero se abrio correctamente
std::getline(s, str)	Lee una linea completa del stream s en el string str
std::string::npos	Constante especial que significa "no encontrado" — devuelta por find()
.find(s, pos)	Busca s en el string a partir de pos
.erase(pos, n)	Borra n caracteres desde posicion pos
.insert(pos, s)	Inserta s en posicion pos
.c_str()	Convierte std::string a const char * — necesario en C++98 para abrir ficheros
LIFO	Last In First Out — el orden de destruccion del stack